TheoryBERT: Final Technical Solution Report

BERT-centered translation and understanding of formal-language objects and their behaviour

Central Problem Statement

The inability of NLP systems to reliably support the translation and understanding of formal-language objects and their behaviour.

TheoryBERT addresses this single problem through one integrated system. Natural-language requirements are interpreted by BERT, translated into formal specifications and executed by explicit mathematical components. DFA, grammar and compiler support are complementary parts of the solution. The web interface makes their structures, execution and verification evidence inspectable.

Complete Solution Scope

The implementation includes a pretrained contextual encoder; a typed semantic-plan decoder with learned literal copying; a structured direct-DFA interface; a grammar-production decoder; exact automata construction, minimization and equivalence; bounded Earley parsing and derivation checking; arithmetic lexing, AST construction and symbolic intermediate code; interactive SVG visualizations; checked Manim export; asynchronous API jobs; model and result caching; versioned datasets; evaluation tools; and deployment packaging.

Final Evaluation

The deployed-model candidate attains 611/641 first-candidate DFA equivalence on the generated test and 86/120 on the frozen fresh-language test. Generated grammar production exactness is 130/130. Mixed CFG fixtures yield 10/14 expected results, and compiler fixtures yield 9/9 including correct rejection. These are separate task-specific measurements. They do not constitute universal English understanding.

Purpose of This Report

This report presents the final technical solution, architecture, algorithms, data, website, implementation and evaluation. It accompanies the eight-page IEEE-style research manuscript. Dataset variants are documented for provenance and reproduction; model-development chronology and before-and-after efficiency comparisons are not the organizing argument.

Contents and Evidence Conventions

Page	Topic
1	TheoryBERT: Final Technical Solution Report
2	Contents and Evidence Conventions
3	Problem Definition and Solution Requirements
4	Literature and Technical Foundations
5	Research Positioning and Key Contributions
6	System Component Architecture
7	Structured DFA Prediction Interface
8	BERT Semantic-Plan Translation Model
9	Literal Copying and Typed Decoding
10	Formal DFA Algorithms and Guarantees
11	External Data and Pretrained Sources
12	DFA Supervision Assets and Dataset Roles
13	Grammar and Challenge Datasets
14	Splits, Leakage and Provenance
15	Training Design and Resource Management
16	Evaluation Methodology
17	Final Translation and Formal-Execution Results
18	Coverage Analysis and System Verification
19	Website and User Experience
20	API, Jobs, Caching and Deployment
21	Grammar Analysis and Checked Derivations
22	Arithmetic Compiler Implementation
23	Evidence Boundaries and Rendering
24	Testing and Integration Verification
25	Reproducibility and Local Operation
26	Limitations and Failure Analysis
27	Integrated Contributions and Research Scope
28	Dataset Fingerprints and Exact Source Links
29	Bibliography
30	Implementation Map and Glossary
31	Expanded Review of the Supplied Literature
32	Supplemental Bibliography
33	Reference Audit: Supplied Items 1-8
34	Reference Audit: Supplied Items 9-15

One problem connects every chapter: reliable NLP support for translation and understanding of formal-language objects and their behaviour. Measured results, implemented capability and future work are distinguished. Verified refers to the explicit formal claim checked, not unrestricted English correctness.

Dataset variants overlap. Fingerprints identify exact artifacts. Corpus counts are not added as independent training data.

Problem Definition and Solution Requirements

One Underlying Problem

The inability of NLP systems to reliably support the translation and understanding of formal-language objects and their behaviour.

The problem concerns the complete connection from an informal requirement to the meaning and behaviour of a formal object. NLP may generate a plausible structure, but a learner must also understand what the structure accepts, how its transitions or productions operate and what evidence supports the answer. A language model response alone does not supply that connection.

For a DFA, the language description determines finite memory, transitions and accepting states. A test word determines only one execution through the machine. For a CFG, the grammar determines possible derivations, while the current token sequence determines the returned parse trees. For arithmetic compilation, the accepted expression determines the AST and symbolic instructions under an explicit precedence grammar. These are different formal manifestations of the same translation-and-understanding problem.

Required Capabilities

Capability	Acceptance criterion
Language interpretation	English requests invoke the trained BERT encoder and the relevant learned head.
Formal translation	Predicted plans or productions produce an explicit inspectable object.
Behaviour	Simulation, derivation and compiler output follow that object and the current input.
Representation	Graph and tree content is computed from current formal data.
Correctness evidence	Each claim states its checker, reference and scope.
Complete product	A web workflow connects models, formal tools, visualizations and deployment.

Design Objective

The proposed system connects interpretation with deterministic formal semantics and evidence-aware explanation. Formal algorithms execute predicted meaning; they do not replace BERT with handwritten English rules. The result is an integrated educational tool with measurable translation quality and checkable execution, rather than a conversational answer detached from a formal object.

The scope is bounded binary regular languages, supported context-free productions and symbolic arithmetic. General English ambiguity, unrestricted CFG equivalence and industrial compiler functionality remain outside the established guarantees. The report keeps these limitations visible while describing every implemented component.

Literature and Technical Foundations

BERT provides contextual language representations that can be adapted to supervised tasks [1]. TheoryBERT uses the pretrained base uncased model as an English encoder, then predicts formal outputs with explicit task heads. The pretrained model card is the source of its artifact lineage and inherited pretraining information [10]. Downstream automata performance is measured in this project rather than inferred from BERT's general NLP results.

Kushman and Barzilay study natural-language regular-expression generation using semantic unification [2]. Their work highlights the value of semantic equivalence when distinct formal expressions mean the same thing. Locascio et al. study learned translation from paired descriptions and regular expressions [3]. Their repository supplies the KB13 mirror used by this project [9]. The local filtered binary task differs from the original source task, so published scores are not used as direct baselines.

The exBERT work studies domain-vocabulary adaptation under limited compute [4]. The repository contains an inspired extension with added embedding rows and a residual contextual layer. Its location and training condition differ from the original architecture, and the V3 experiment also fine-tunes two BERT blocks. The semantic plan head freezes that V3 encoder. The report keeps these conditions distinct.

Herzig et al. examine intermediate representations for compositional semantic parsing [5]. The present plan experiment applies the general design motivation to automata. Oren et al. analyze contextual representations, alignment and program-template balance [8]. These findings motivate literal-copy supervision and operator-specific analysis here. The implementation does not reproduce either paper's full model, data or experimental controls.

PICARD demonstrates formal constraints during autoregressive output generation [6]. TheoryBERT instead predicts plan positions in parallel and uses a small typed prefix beam afterward. The shared idea is formal admissibility during decoding; no PICARD code or SQL parser is used. A typed output remains capable of expressing the wrong meaning, so structural validity is evaluated separately.

CheckList motivates targeted behavioral tests beyond aggregate held-out accuracy [7]. The local challenge suites probe wording, boundaries, negation and composition. Cornell's compiler notes support the classical connection from regular specifications to total automata and lexical processing [12]. NLTK supplies the Earley parser used for context-free derivations [11]. These are established foundations, not new formal algorithms introduced by this project.

The research question is how contextual NLP can support translation and understanding through explicit formal targets, executable semantics and inspectable behaviour. Prior work supplies the representation and parsing foundations; this system combines them in a complete educational workflow and evaluates each boundary with task-appropriate evidence.

Research Positioning and Key Contributions

BERT as the Interpretation Basis

BERT processes the English requirement and supplies contextual representations to the formal-output heads. The plan head learns operators, numeric arguments, composition and literal-copy locations. The grammar head learns production tokens. A separate structured DFA interface supports direct transition prediction. No hidden phrase matcher constructs replacement answers in the learned path.

Executable Semantics as the Bridge

A model-produced ENDS(0101) plan has a defined meaning that the automata compiler can execute exactly. A model-produced conjunction composes two formal languages through reachable product construction. If the interpretation is wrong, the exact compiler still constructs the wrong intended language. This is why semantic reference checks and honest candidate labels are necessary alongside structural validity.

Contribution 1: Integrated Formal Translation

The system connects contextual language analysis with typed regular-language plans and grammar productions. Learned span copying preserves literal identity and leading zeros. Shared encoder loading supports multiple tasks without requiring a separate English interpreter in every workspace.

Contribution 2: Behavioural Understanding

The interface displays actual machine states and transitions, current input traces, grammar derivations, arithmetic syntax trees and symbolic code. It connects abstract objects with concrete behaviour while preserving the distinction between a language graph and one execution specimen.

Contribution 3: Checkable Claims

Independent DFA equivalence, derivation checks, AST/IR consistency and sealed export evidence make correctness claims inspectable. The application distinguishes semantic equivalence, structural validity and valid execution under a predicted grammar.

Contribution 4: Complete Reproducible System

The implementation includes versioned supervision, canonical-language partitioning, public source attribution, model fingerprints, guarded adaptation, API jobs, caching, responsive workspaces and a hash-checked deployment bundle. The contribution is their integrated solution to the stated problem. Individual neural and formal techniques build on prior work.

System Component Architecture

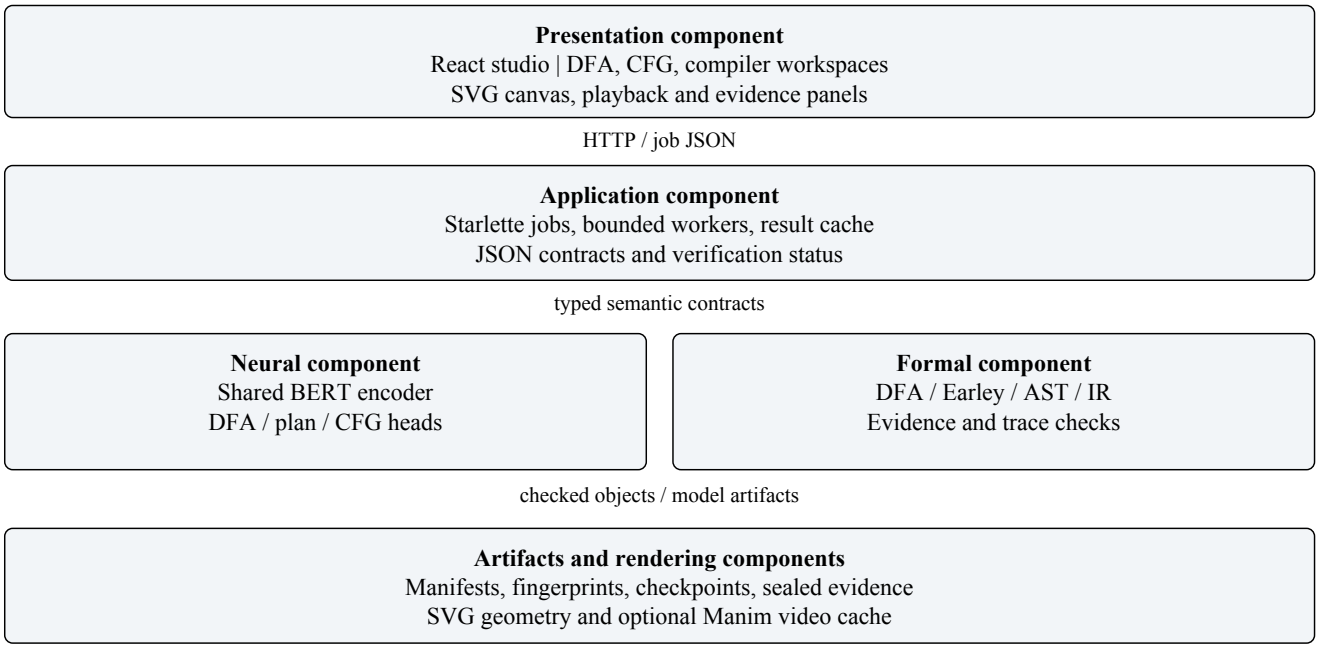


Figure 1. Component/deployment architecture and named formal interfaces.

Component	Responsibilities and contracts
React presentation	Description and input editors; settings; graph/tree/AST playback; evidence and research panels.
Starlette application	Validates request shape, resolves active model, manages bounded jobs, retains result/evidence records.
Neural subsystem	Shared immutable encoder; direct DFA head; grammar token decoder; typed plan and literal-pointer head.
Formal kernel	DFA JSON checks, construction, simulation, minimization, equivalence; CFG parsing and tree checks; AST and IR consistency.
Artifact store	Pinned source metadata, dataset manifests, model configs, hashes, training logs and evaluation outputs.
Rendering subsystem	Graph/tree geometry, browser SVG and optional sealed-evidence Manim exports.

The main interfaces are HTTP job JSON between browser and application, typed DFA/grammar/plan objects between model and formal kernel, and checked objects plus traces between formal kernel and rendering. The component view intentionally avoids procedural decision diamonds or an implied single execution sequence. Optional preview, manual-entry and reference-verified paths use the same mathematical contracts with different provenance.

A head checkpoint names the encoder path and SHA-256 fingerprint. Loading rejects a mismatch. Model caching shares the compatible encoder between plan and grammar tasks, while each task retains its own small head. This avoids a second full BERT copy for an ordinary task switch. The application remains the authority for verification status and renderable evidence.

Structured DFA Prediction Interface

Representation and Implemented Heads

The structured DFA interface predicts a machine directly from a pooled BERT representation. Separate heads predict active state count, accepting-state bits and transition destinations for each supported state and binary symbol. The maximum state count is eight. Training masks inactive positions and inference constrains destination indices to the active range.

Formal Output Contract

The predicted object contains a declared state set, alphabet, start state, accepting set and total transition map. Structural validation rejects undeclared destinations, missing symbols, duplicate states and invalid accepting roles. Canonicalization removes unreachable structure and minimizes the machine for semantic identity and comparison.

Bounded Candidate Search

The first candidate takes the highest-ranked neural decisions. Bounded alternatives explore uncertain count, accepting-bit and transition choices through score-gap search and deduplicate completed machines. References do not generate candidates. When a reference is supplied, exact equivalence identifies a matching candidate; without one, the first candidate remains an explicitly unverified preview.

Relationship to the Semantic-Plan Model

The final default regular-language route predicts typed plans and then constructs machines. The direct interface remains implemented for explicit selection, controlled research and compatibility with the same DFA contract. Both use BERT for language interpretation and both connect to the same verifier, simulator and visualization. This is an architectural choice between output representations, not a separate user problem.

Direct prediction can describe small arbitrary transition tables but couples many output decisions. Typed plans express supported families compositionally and expose literal arguments. Neither representation guarantees correct English interpretation. The downstream checker establishes only the claim permitted by available evidence.

Encoder and Adaptation

The shared evaluated encoder starts from google-bert/bert-base-uncased. Its domain extension supplies additional token embeddings and a residual contextual module; the recorded encoder adaptation updates the top two BERT blocks while preserving protected lower weights. The semantic-plan and grammar heads bind to this exact checkpoint through fingerprints.

A supported task head must load a compatible encoder. Changing the backbone without retraining and evaluation invalidates that compatibility. Checkpoint dispatch preserves these boundaries while keeping model selection independent of the final visual representation.

BERT Semantic-Plan Translation Model

Architecture

The semantic head has 537,385 trainable parameters. It uses immutable token features from the evaluated V3 encoder, projects width 768 to 128 and applies two Transformer decoder layers. Each layer has four attention heads, feed-forward width 256 and dropout 0.1. Twenty-four learned queries represent maximum output positions. A linear head predicts typed prefix tokens, and two projections supply start/end pointer scores over input tokens.

The model is parallel rather than autoregressive: every output query attends to contextual input memory, and query self-attention lets output positions interact. The head does not consume a handcrafted English parse. Its loss supplies supervision for operator identity, symbolic arguments, output termination and copied literal boundaries. Encoder computation uses inference mode and does not update any BERT parameter.

Plan Vocabulary and Scope

Operators	Meaning
ALL, NONE	Accept every binary word or no binary word.
STARTS, ENDS, CONTAINS, EXACT	Prefix, suffix, contiguous substring or one complete literal word.
NOT, AND, OR	Complement, intersection and union of regular languages.
COUNT_EQ / LE / GE	Equality or inclusive bound on occurrences of one symbol.
COUNT_MOD	A symbol count leaves a given remainder modulo a positive modulus.
LENGTH_EQ / LE / GE / MOD	The analogous total-length requirement.

The typed schema distinguishes subexpressions, patterns, symbols and integer arguments. Implementation validation permits literals of length 1 through 32, integers 0 through 16, positive moduli with smaller nonnegative remainders, bounded nesting and at most 64 final DFA states. The training catalogue uses patterns through length eight and moduli 2 through 8. Extrapolation to the full implementation bounds has not been established.

Supervision and Objective

Targets are prefix programs with an EOS marker and padding. Single-bit patterns use learned fixed classes. Longer literals use COPY plus start/end token labels. The loss is token cross-entropy plus 0.6 times start-pointer cross-entropy plus 0.6 times end-pointer cross-entropy at copy positions. Padded token positions and non-copy pointer positions are excluded. Invalid or unaligned supervision raises an error rather than being silently repaired.

The encoder fingerprint binds the head to the exact V3 weights used for feature extraction. A different encoder requires another compatible training/evaluation condition. The small checkpoint stores head weights and configuration without copying the full backbone.

Literal Copying and Typed Decoding

Why Literal Copying Matters

Binary patterns can have leading zeros, repeated fragments and lengths unseen in a particular training subset. A model that chooses from a fixed catalogue of literal strings cannot faithfully support new literals. The pointer head instead scores a start token and an end token, allowing the predicted COPY action to reproduce the exact original substring. The interpretation of what the pattern means still comes from learned operator tokens.

Candidate span extraction checks token offsets and raw characters. A valid copied pattern is a complete contiguous binary span, not a subword cut from the middle of a larger bit literal. It preserves the original text exactly, including leading zeros. This is a type and boundary constraint, not an English keyword rule. The neural pointers choose among available literals when a request contains several of them.

Bounded Prefix Search

A beam item records accumulated score cost, predicted prefix values and a stack of argument types still required. At an expression position it permits the supported operators; at a pattern position it permits single-bit classes and COPY when a valid span exists; symbol and number positions have their respective finite vocabularies. Composite operators push their argument types. A completed stack requires an EOS prediction before the tree is accepted.

Each state expands at most three typed token alternatives. COPY expands at most two highest-scoring spans. Beam width is eight and maximum plan length is 24. The search limits compositional nesting, and the compiler independently validates the completed tree. Scores use gaps relative to the strongest token at each position and relative span scores, avoiding an unexamined preference for short one-state plans. Equivalent completed DFAs are deduplicated.

Failure and Abstention Behavior

The compiler rejects an invalid modulus, a remainder outside its range, unsupported arguments, excessive construction size or excessive final state count. If no valid plan remains within the bounded search, the predictor reports an error. It does not fall back to phrase matching or retrieve a benchmark answer. A syntactically valid but semantically wrong plan remains a wrong prediction and is scored accordingly.

The decoder receives no gold DFA during inference. The independent reference is only available to evaluation or the application verifier after candidates are produced. This boundary prevents reference-assisted construction from inflating first-candidate accuracy. Within-five success is intentionally a separate, reference-assisted metric.

Example Contract

For a predicted AND(CONTAINS("00101"), COUNT_MOD("1", 2, 0)), the formal compiler must preserve the literal and enforce an even count of ones. Unit tests inject ideal neural logits to verify exact copying and tree reconstruction independently of training. Separate predicate tests enumerate short words to check that compilation matches conjunction, negation, counts and length conditions. These tests validate implementation mechanics, not the trained model's understanding of arbitrary English.

Formal DFA Algorithms and Guarantees

Structural Validation and Simulation

A DFA requires a nonempty unique state set, explicit alphabet, declared start and accepting states and exactly one declared destination for every state/symbol pair. Strict JSON loading rejects duplicate keys before they could conceal contradictory rows. Simulation follows transitions from the start state and records each consumed symbol, source and target. The final accepting status is computed from the machine, not inferred from the animation.

Construction from Plans

Pattern constructors encode relevant suffix/prefix memory, including overlap. Exact words and count/length bounds use the supported regular-expression compiler. Modular requirements use residue states. Complement changes accepting roles on a complete DFA. Intersection and union explore reachable pairs of component states and apply the corresponding Boolean accepting condition. An intermediate 256-pair construction budget and a 64-state final budget constrain the new plan path.

The regex compiler has a deliberately supported dialect and explicit repetition/state limits. It is not a drop-in implementation of every PCRE feature. The public dataset import therefore filters unsupported constructs and checks accepted binary restrictions with bounded word comparisons. Formal source labels and filtering decisions are retained for audit.

Reachability, Minimization and Canonicalization

Reachability removes states inaccessible from the start. Partition refinement begins with accepting/nonaccepting groups and repeatedly separates states by destination-group signatures. Canonicalization traverses the minimal reachable DFA in fixed alphabet order and assigns q0, q1 and subsequent names. This yields a stable language identity for equivalent complete binary DFAs. The implementation is a straightforward refinement procedure, not a claimed optimal Hopcroft implementation.

Exact Equivalence and Counterexamples

Product BFS starts from the candidate/reference start pair. If their accepting memberships differ, the predecessor map reconstructs a shortest distinguishing input. Otherwise it explores successor pairs until exhausted or a product budget is reached. Exhaustion without disagreement establishes exact equivalence over the matching alphabet. Empty input is represented by an empty symbol list; it must not be confused with absence of a counterexample.

For complete machines with matching alphabets, exploration is bounded by the product of state counts times alphabet size. Minimization and a readable graph are useful analysis operations but do not establish equivalence to English. A formal reference can be supplied manually or by an independent benchmark fixture. Without it, the system can validate and simulate the candidate while withholding an equivalence claim.

The same minimized object feeds graph geometry, transition tables, access strings, completion strings and trace inspection. Keeping these views derived from one object prevents inconsistencies between a state counter and the actual canvas.

External Data and Pretrained Sources

Pretrained Encoder Artifact

Source: google-bert/bert-base-uncased on Hugging Face

The model revision is 86b5e0934494bd15c9632b12f734a8a67f723594. The local source manifest records download metadata and SHA-256 values for artifact files. The pretrained weights file fingerprint is 68d45e234eb4a928074dfd868cead0219ab85354cc53d20e772753c6bb9169d3. The model artifact is Apache-2.0 licensed. The project reuses its contextual English knowledge and adapts it through downstream supervised tasks.

The model card identifies BookCorpus and English Wikipedia as original pretraining sources. They are inherited lineage, not locally downloaded training datasets in this project. This report therefore does not add billions of pretraining words to the supervised project-data count or imply that the team performed large-scale BERT pretraining. Source-card limitations and biases also remain relevant to downstream interpretation.

KB13 Description/Regex Pairs

Source: pinned DeepRegex KB13 mirror

The downloaded source and target files contain 824 aligned rows. The local import accepts 55 after binary-alphabet restriction and supported-dialect checking. It rejects 497 unsupported-dialect rows, 271 constant languages after binary restriction and one excessive repeat bound. Subsequent corpus deduplication leaves 47 public-regex-labelled descriptions in the V3 version. Fifty-five imported rows and 47 retained rows describe different stages and are not contradictory.

The supplied datasets/license.md is an MIT license with 2016 Nick Locascio copyright. Its notice must accompany redistributed source material as required by that license. The report links the actual pinned files in the source appendix. The project does not claim to train on the full KB13 benchmark or the larger DeepRegex synthetic corpus. Other popular regex/semantic-parsing datasets are literature context only unless explicitly listed as a local input.

Filtering and Audit Limits

Restricting a regex to binary strings can substantially change its language. A source expression about letters may become empty, universal or otherwise irrelevant after restriction. Constant-language removal limits this artifact. Supported conversions are checked against Python regex behavior on short binary words, but bounded import checks are not presented as a proof of equivalence for all regex dialect features. Unsupported features are rejected instead of guessed.

The local manifest records exact URLs, byte lengths and hashes for descriptions, regex targets and license. Generated corpus manifests then record the resulting rows and filtering counts. This two-level provenance distinguishes the external bytes from the project's derived supervision.

DFA Supervision Assets and Dataset Roles

Local dataset	Rows / languages	Train / validation / test
data/examples/sample.jsonl	261 / 96	181 / 39 / 41
english-dfa.jsonl	1,785 / 208	1,363 / 213 / 209
english-dfa-expanded.jsonl	3,207 / 208	2,437 / 391 / 379
english-dfa-v3.jsonl	6,275 / 696	4,889 / 745 / 641
english-dfa-robust.jsonl	6,738 / 1,468	4,924 / 1,173 / 641
english-plan-v1.jsonl	11,415 / 2,914	9,187 / 1,171 / 1,057

Table 1. Generated DFA versions live under data/generated/ unless a path is shown. They overlap.

The dataset registry covers every implemented supervision asset. The shipped sample supports mechanics and canonical splitting. English DFA v1 and expanded DFA provide semantic template supervision over 208 language identities. V3 supplies 6,275 rows over 696 identities for the shared adapted encoder. Corpus names identify reproducible input artifacts rather than stages in a claimed accuracy trajectory.

V3 family counts are 1,351 suffix, 1,338 prefix, 1,325 substring, 864 count-modulo, 504 exact-literal, 424 length-modulo, 236 bounded-count, 180 avoids, 47 public-regex and six edge-case descriptions. These counts describe rows rather than balanced language families. Several descriptions can express the same canonical language. A high row-weighted result can therefore differ from a language-balanced result.

The 6,738-row robust DFA asset supplies additional wording and protected challenge groups for a separate direct-head training condition. It retains the 641-case generated regression partition. It is included for complete data provenance and reproducibility; it is not the final semantic-plan training corpus.

The plan version annotates 6,274 of the 6,275 V3 descriptions with an equivalent copyable typed plan. One description has no supported copyable annotation. Additional plan-generated descriptions and eligible supplementary challenge rows yield 11,415 total examples and 2,914 canonical languages. Only the 9,187 training rows contribute gradients; 1,171 validation rows select the head. The 1,057 plan-file test rows are retained for a separate audit and are not the frozen v2 comparison reported here.

A plan label is accepted because its compiled DFA has the same canonical identity as the gold DFA and its required multi-bit literal appears in the description. This is offline supervision generation. At inference, the model does not search that catalogue for an English match. Many formal plans can denote one language; label selection favors a shorter stable equivalent representation, while evaluation remains language based.

Local generation code supplies the construction recipe. Generated JSONL files stay out of Git history. The companion dataset archive supplies corpus variants, grammar files, pinned KB13 input/license and a file/hash register. The archive is committed with the reports, while large model tensors are distributed as a separate versioned GitHub Release.

Grammar and Challenge Datasets

Dataset / role	Examples	Partition counts
Original grammar training corpus	1,299	999 train, 170 val, 130 test
Robust grammar augmentation	456	371 train, 85 validation
Robust merged grammar corpus	1,652	1,267 train, 255 val, 130 test
Mixed challenge v1	165	71 dev, 46 val, 48 test
Plan comparison v2	254	134 validation, 120 test
Shipped manual challenge fixtures	3	challenge only

The original grammar generator covers matching counts, balanced delimiters, ordered blocks, palindromes and arbitrary words over selected symbols from a, b, x, y, 0 and 1. It also includes canonical arithmetic grammars as known fixtures. Its 1,299 examples contain 123 unique production identities. The model is a learned production-token decoder, not a retrieval table of English patterns.

The robust augmentation contributes 456 descriptions. Merging includes deduplication and removal of protected training identities, so 1,299 plus 456 is not the final row count. The robust dataset has 1,652 rows and 119 grammar identities. Arithmetic, ambiguous addition and compiler grammars are intentionally known training fixtures. Their product behavior is tested, but it is not held-out grammar discovery.

The original grammar dataset is stored at artifacts/bert-grammar/dataset.jsonl. A pooled-feature pilot dataset also exists at artifacts/bert-grammar-pilot/dataset.jsonl. The robust merged file is artifacts/grammar-robust/dataset.jsonl. The augmentation is data/generated/grammar-robust.jsonl. These are project-generated corpora without a separate external dataset license or public download URL.

Mixed challenge v1 contains 100 DFA, 38 CFG and 27 compiler requests. Its reserved subset contains 23 DFA, 14 CFG and nine compiler cases. It covers wording interpretation, formal structure, unsupported bounds and downstream visualization. Because it is inspected and eligible development cases inform supervision, its outcomes are regression evidence rather than an untouched independent test.

Plan comparison v2 contains 254 unique formal language identities outside the new plan training split. Its descriptions use a distinct wrapper and selected held-out wording styles. Validation has 134 cases; test has 120. Targets include cases above the direct head's capacity, so the analysis also reports a matched subset with at most eight states and no more than 64 baseline tokens. The file is frozen by SHA-256 before either model predicts it.

The small shipped challenge file has three manually selected demonstrations, excluded from the original sample splits. A separate suffix-01 JSON object is an independent reference fixture, not another training corpus. Public links for generator code and the exact local data register appear later in this report.

Splits, Leakage and Provenance

Canonical Language Grouping

DFA examples are grouped by the canonical minimized reachable machine before splitting. All paraphrases of an equivalent language stay together. This is stronger than random description splitting, which can place the same mathematical answer in both training and test. The plan dataset inherits all known prior language roles and assigns genuinely new identities by a deterministic hash rule. Assertions reject a language identity that appears in multiple partitions.

A single role-resolution function assigns each canonical language to its protected partition across generated and supplementary descriptions. Build assertions reject conflicting identities before writing a corpus. Tests verify that the frozen evaluation languages do not occur in plan training. These controls make split integrity an executable requirement.

What Is Still Shared

Language-group protection does not imply unseen linguistic constructions. Generated families share operator concepts, vocabulary and some template patterns. The v2 wrapper differs from training descriptions, while its underlying clause families remain generated. The evaluated V3 encoder has its own supervised history, but old held-out formal language groups remain excluded from its gradients. The frozen evaluation is designed as a local controlled test, not a claim of independently sampled real-world requests.

Exact duplicate descriptions are removed when building the plan corpus. Multi-bit targets must be copyable from the description. New comparison descriptions are checked against training descriptions, and their canonical language identities are checked against the training set. The comparison manifest records exact bytes. A second experiment should create another comparison version rather than overwrite v2 after observing errors.

Selection and Test Boundaries

Internal validation selects the head using plan-and-copy exactness, with loss as a tie-break. Decoded validation and known product fixtures check application compatibility. Reserved test outcomes are recorded after selection and are excluded from retraining and checkpoint selection in the reported evaluation.

The V3 test has already been evaluated across project iterations and is now a regression benchmark. The mixed challenge test is also already inspected. Results from those sets remain useful, but repeated reuse reduces their independence. A report must identify that status rather than repeatedly call the same known set unseen.

Artifact Identity

Each important corpus has a SHA-256 fingerprint. A head config records its dataset fingerprint and exact encoder-weight fingerprint. Active-model pointers name selected artifacts. Evaluation outputs include per-case results and protocol metadata. Hashes prevent silent substitution of bytes; they do not establish absence of every conceptual leakage source. Reproducibility additionally requires source version, dependency versions, random seeds and access to the local generated artifacts.

Training Design and Resource Management

Supervised Adaptation

The final semantic head learns typed prefix tokens and literal span pointers from language-plan pairs. Its objective combines token cross-entropy with weighted start and end pointer losses at copy positions. The head contains 537,385 trainable parameters. BERT token features are frozen during this adaptation, and the trained head records the exact encoder fingerprint.

Data Separation

Equivalent DFA languages are canonicalized and grouped before partitioning. Protected split assignments keep one canonical language in one partition. Validation selects the checkpoint; test inputs do not enter the feature cache, gradient updates or checkpoint-selection metric. Grammar supervision is grouped by canonical production identity under its declared normalization.

Bounded Feature Storage

Feature extraction stores only real tokens in a packed disk-backed float16 cache. Small batches are padded on demand. This representation limits cache storage and avoids a dense array sized by maximum token length. The cache is deleted after completed adaptation. Final inference still uses the original evaluated FP32 encoder and head tensors.

Local System Controls

The training launcher runs one heavy job at a time with two CPU threads and reduced scheduling priority. Resource guards record available disk, available memory and swap. The configured stopping conditions preserve 3 GiB free disk, an 8% available-memory floor and a 0.75 GiB allowance for swap growth. These controls reduce resource pressure but cannot prevent unrelated operating-system changes.

Shared Inference and Storage

DFA and grammar heads share one compatible immutable encoder in the web process. Small task checkpoints store their heads and configuration without duplicating the backbone. Model weights remain outside Git history and are distributed as a versioned release asset. CPU memory-mapped loading uses meta initialization and assigned file-backed tensors, preserving FP32 values without quantization.

Measurement Interpretation

The recorded semantic-head adaptation takes 230.8 seconds including feature extraction. Trainable parameter count, optimizer-state cost, inference latency and translation quality describe different properties. They are documented for reproducibility and operational planning. The research problem remains formal translation and understanding, rather than an efficiency-improvement narrative.

Evaluation Methodology

Questions Evaluated

The final system is evaluated for translation accuracy, faithful formal execution and faithful presentation. Translation asks whether a predicted object denotes the intended supported language. Execution asks whether traces, derivations and symbolic code follow the object. Presentation asks whether the web view corresponds to the current object and input. These checks assess three boundaries of the same central problem.

Automata Metrics

First-candidate exact equivalence is the primary DFA translation metric. A complete product exploration compares the candidate with an independent reference and returns a shortest distinguishing word on disagreement. Unsupported or invalid predictions count as incorrect. Within-five success reports whether a correct candidate exists among a bounded set and requires a reference for selection.

Grammar and Compiler Metrics

Generated grammar tests use canonical production exactness. Runtime derivations are independently checked against the displayed productions and terminal yield. Compiler fixtures test expected acceptance or rejection and consistency between a checked parse, AST and symbolic IR. These measurements cannot be collapsed into a single product-wide accuracy percentage.

Partitions and Scope

The plan corpus contains 11,415 examples and 2,914 canonical languages. The frozen wording suite has 134 validation and 120 test cases outside training. The generated DFA regression test has 641 cases. A 57-case fresh subset obeys the direct structured interface bounds. Mixed CFG and compiler fixtures are inspected regression evidence and are labelled accordingly.

Separation of Selection and Reporting

Checkpoint selection uses validation and known product fixtures. Test outcomes are reported after selection. Tests that have influenced model design are not described as untouched future evidence. Per-case records preserve source descriptions, targets, state counts, candidate outcomes, errors and timing.

Application Validation

Unit tests assess formal algorithms and model contracts. HTTP tests exercise real job submission, provenance, changed inputs, invalid requests and video gates. Browser tests compare visible graph/tree content with server results and check responsive layouts. The deployment workflow separately builds the full inference image and exercises it under a declared memory limit.

Final Translation and Formal-Execution Results

DFA Translation

Test asset	First correct	First equivalence	Within five
Generated DFA regression	611/641	95.32%	97.66%
Frozen fresh languages	86/120	71.67%	99.17%
Capacity-matched fresh subset	38/57	66.67%	100%

These are final-model measurements, each tied to a particular evaluation asset. The generated test is useful for established supported families; the fresh suite probes wording outside training languages. The gap between these results makes unrestricted-language claims inappropriate.

Grammar Translation

The generated grammar test contains 130 examples with 130 exact canonical production outputs. The mixed CFG fixture subset yields 10 successful expected outcomes out of 14. These values evaluate different distributions and evidence types. A production-exact score does not prove equivalence between arbitrary grammars or general understanding of English specifications.

Compiler Outcomes

The compiler fixture subset yields 9 expected outcomes out of 9, counting valid-expression processing and correct rejection. The arithmetic implementation supports identifiers, numbers, parentheses, unary signs and the standard additive and multiplicative operators. It is a symbolic educational compiler path, not a complete industrial compiler.

Formal Evidence

DFA equivalence is an exact claim about candidate and reference languages. A checked derivation establishes that the displayed grammar derives the selected token sequence. AST and IR consistency establishes that the symbolic lowering preserves the parsed expression under the implemented arithmetic contract. The claims differ and remain labelled separately in the interface.

Interpretation

The evaluation supports a useful bounded language-to-formal solution with inspectable execution. It does not justify guaranteed correctness for arbitrary descriptions. Where an independent reference is absent, the candidate preview helps the user inspect the interpretation, but remains distinct from a proved translation.

Coverage Analysis and System Verification

Operator Coverage

Fresh-test family	First correct	Interpretation
Exact word	23/23	Literal identity and whole-word condition.
Contains pattern	21/21	Contiguous substring condition.
Starts with pattern	20/22	Prefix interpretation.
Ends with pattern	1/18	Substantial wording weakness remains.

The suffix result is a material limitation of the final model under the frozen fresh wording. A supported operator in the formal compiler is not a guarantee that its English description is recognized reliably. The report retains this finding because an aggregate score can obscure the behaviour most relevant to a particular user request.

Software Verification

The final unit suite contains 105 passing tests and the React production build succeeds. The same software checks pass in GitHub Actions. Model-loader tests compare every tensor and output between copied and memory-mapped loading. Installer tests cover exact bytes, archive corruption, unsafe paths, unlisted members and symlink escapes.

End-to-End Behaviour

The local deployment check exercises seven cases: BERT suffix translation against an independent five-state reference; rejection after changing the input; an unreferenced preview without false equivalence; BERT grammar generation and checked parsing; BERT arithmetic grammar and AST; invalid expression rejection; and real MP4 export from verified evidence.

Visualization Fidelity

The visible graph follows the formal state set, complete transitions, start state and accepting set. Input traces determine highlights and acceptance. Grammar derivations and compiler syntax trees come from checked structures. Presentation components are reused across model interfaces, so the display does not become a substitute language interpreter.

Limits of the Evidence

Passing software tests confirms implemented contracts on exercised cases. It does not raise the semantic score of a model or establish user learning outcomes. A cloud container check establishes behaviour under its measured memory budget. It does not by itself prove internet-scale reliability or indefinite availability.

Website and User Experience

Workspaces and Technology

The primary interface is a React application built with Vite and served from frontend/dist by Starlette. React Router supplies workspaces and navigation. The source uses Lucide icons, Motion interactions and locally packaged fonts. A legacy Streamlit application remains in the repository, but the current local launcher serves the React product. The backend uses Python, PyTorch, Transformers, NLTK, NumPy and optional Manim rendering.

DFA Interaction

The user enters a language description and a binary test string. Learned mode invokes the active evaluated BERT checkpoint or an explicitly selected local checkpoint. Reference demonstrations, intentionally incorrect fixtures and manual DFA JSON are distinct modes with clear provenance. A benchmark example can supply its independent reference; a free-form request generally cannot. The interface shows either verified evidence or an explicitly unverified candidate preview.

The visible graph is generated from actual states and transitions. Accepting-state rings, start markers and edge labels follow the formal object. Simulation highlights the current state and consumed input symbol. Input-only edits recompute the trace for the same machine, while a changed English requirement triggers a new prediction. A suffix-01 requirement produces a three-state minimal machine; suffix-0101 requires five. A longer test word can loop through those five states without creating extra graph nodes.

Analysis panels derive transition tables, state visits, access strings, accepting completions and accepted/rejected examples from the candidate. These facts help inspect behavior but do not prove correspondence to English. A shortest formal counterexample is shown when an independent reference reveals a disagreement. Empty input is handled as epsilon and is not replaced by an arbitrary demonstration string.

Grammar and Compiler Interaction

The grammar workspace displays BERT-produced productions and parse outcomes for the current tokens. Input changes can reuse the same compatible BERT grammar result, then reparse the new sequence. Tree alternatives and derivation playback correspond to actual parser output. Arithmetic mode additionally presents lexemes, a parse structure, AST and symbolic IR. Unsupported grammar output or invalid source is reported without substituting a known fixture.

Presentation Preservation and Responsive Behavior

The evaluation keeps Diagram.jsx, layout.py, scenes.py, renderer.py, styles.css and analysis.css byte-identical to their recorded fixed presentation hashes. Model provenance text is updated where necessary, while the final canvas and playback stay visually consistent. Browser checks exercise desktop and mobile widths, verify that the graph changes with a changed language and that acceptance changes with the test string. The design goal is stable presentation with correct dynamic content.

API, Jobs, Caching and Deployment

Route	Role
GET /api/health	Server readiness, active local model and render capability.
GET /api/catalog	Examples, model choices and grammar availability.
GET /api/research	Measured summaries and experiment metadata.
POST /api/jobs	Submit bounded DFA/CFG/compiler/render work.
GET /api/jobs/{job_id}	Poll progress, result or explicit error.
POST /api/simulate	Recompute a DFA trace from a prior result.
GET /api/videos/{name}	Serve an existing validated cache filename.

The server uses a bounded ThreadPoolExecutor. Local defaults are two workers and at most four running jobs. Result retention is bounded at 128 entries and job history at 256. Request size, description size and input limits prevent accidental unbounded work. Hosted environment variables can reduce workers and job counts. Parsing uses a configurable subprocess timeout rather than allowing an unbounded grammar request to block indefinitely.

Checkpoint dispatch reads config.kind. A semantic_plan checkpoint loads head.pt and its named encoder; direct checkpoints load weights.pt. A shared encoder cache is keyed by resolved path and file revisions. Grammar and plan heads validate their encoder SHA-256 before use. The active-model pointer is explicit; finding an arbitrary folder does not automatically make it the default. Smoke checkpoints are rejected from the learned product route.

Model-generated language_plan metadata is preserved with a DFA result when available. The trace and diagram still consume the ordinary DFA contract. A reference comparison can select a later candidate, so metadata is associated with the selected candidate rather than blindly reporting the first plan. No model state is stored as a mutable last-plan field shared across concurrent requests.

The deployment image builds React and installs the CPU inference and rendering runtime. A versioned GitHub Release distributes the original FP32 encoder and compatible heads. The installer verifies archive and member hashes against deploy/model-bundle.json. Startup loads both evaluated active checkpoints and fails if either is unavailable. The health response reports model readiness and the Render source commit. A constrained cloud container test separately checks the hosting memory budget.

The research endpoint adapts the active model's measured original regression result to the existing frontend metric contract and exposes the complete plan comparison separately. This keeps displayed statistics tied to the selected architecture. The UI text identifies the results as a measured regression set rather than continuing to describe a repeatedly inspected set as unseen.

The server treats submitted source and descriptions as data. The arithmetic compiler does not invoke Python eval, shell execution or a generated runtime. The product is a local educational application, not an authenticated multi-tenant inference service. Internet-facing hardening, persistent accounts and distributed job queues are outside the implemented scope.

Grammar Analysis and Checked Derivations

BERT Grammar Decoder

The grammar path uses the evaluated V3 token encoder and a separately trained two-layer cross-attention production decoder. It predicts a finite grammar-token vocabulary and EOS. It does not select a handwritten grammar by matching English text. The head config records vocabulary, sequence length, encoder fingerprint and dataset fingerprint. Invalid grammar output raises an explicit prediction error with the actual candidate productions retained for inspection.

The robust head is trained on 1,652 generated/augmented rows. Its internal validation exactness and generated test reflect canonical production targets, not arbitrary grammar-language equivalence. Known arithmetic forms are intentionally present in training. This makes the compiler workspace reliable for its intended grammar while preventing those fixtures from being advertised as novel held-out discovery.

Parsing and Evidence

The formal CFG component parses grammar text, checks declared productions, validates terminals and runs NLTK Earley parsing within a subprocess budget. Returned trees are bounded in count. A tree is checked by matching each expansion to an actual grammar production and comparing the terminal yield with the input token sequence. Epsilon productions are represented explicitly in derivation handling.

A successful tree proves that the supplied token sequence is generated by that predicted grammar. It does not prove that all strings generated by the grammar satisfy the English requirement. General CFG equivalence is not implemented. Bounded challenge membership checks are useful tests, but they must not be labelled a complete equivalence proof.

Input Dependence and Reuse

A grammar represents a language, so a change from ab to aabb should not require a different grammar for matching counts. The result cache can reuse BERT productions when description, model checkpoint and head/encoder revision match. It reparses the new token sequence and regenerates its tree layout. A new model revision invalidates the reuse. This saves encoder work while preserving correct input-dependent content.

The parser can reveal several derivations for an ambiguous grammar. The interface lets the user inspect alternatives and expansion order, with a clear bound on returned trees. One returned tree does not establish unambiguity. Arithmetic grammar precedence is supplied by productions, while the ambiguous addition fixture intentionally exposes alternative groupings.

Grammar Analysis Outputs

The analysis derives terminal/nonterminal sets, production information, nullable behavior and other inspectable facts from the actual grammar. Tree labels and positions come from returned nodes, not a generic template. Character, token and arithmetic-lexeme modes have different tokenization contracts; a mismatch is an input issue rather than evidence that the model should silently reinterpret source.

Arithmetic Compiler Implementation

Supported Language

The compiler supports symbolic arithmetic expressions with identifiers, numeric literals, parentheses, binary plus/minus/multiplication/division and unary signs. The intended BERT-produced grammar gives multiplication/division higher precedence and binary operators left associativity. Identifiers represent external inputs. The system does not provide statements, declarations, types, loops, functions, control flow, machine code or a runtime interpreter.

Lexing and Concrete Tokens

The lexer records token category, original lexeme and source offsets. It distinguishes identifiers, numbers and punctuation, rejects unsupported characters and preserves decimal literals. It accepts at most 4,000 source characters and 64 arithmetic tokens. Source is treated as data. An invalid sequence such as a+*b fails parsing under the actual predicted grammar rather than being fixed into another expression.

From Parse Tree to AST

The compiler checks that terminal leaves align exactly with the lexed categories. It maps those leaves back to their original identifier or number lexemes. Unary signs create unary AST nodes. Binary operators create ordered children, and parenthesis productions preserve grouping without becoming redundant AST nodes. Unsupported production shapes raise an error instead of assuming a familiar arithmetic grammar.

For (total + 3) * scale - 4 / 2, the AST must preserve the parenthesized addition, multiplication by scale, division of 4 by 2 and the final subtraction order. This is a structural translation from a checked derivation. It is not a claim that the system infers arbitrary compiler semantics from a broad English programming-language specification.

Intermediate Representation and Independent Check

Postorder lowering produces temporaries t0, t1 and subsequent targets. Each instruction records an operator and ordered operands, which may be identifiers, numbers or earlier temporaries. The result identifies the final value. An independent reconstruction checks temporary-before-definition errors, operator arity and operand order, then compares the reconstructed symbolic expression with the AST expression. This catches lost or reordered subexpressions in lowering.

The compiler view displays tokens, AST and instruction sequence derived from the current source. Changing source alters these structures. A simple leaf-only expression can have no intermediate instructions and no expansion animation steps; that is valid behavior, not a missing diagram. Division is kept symbolic, so runtime division-by-zero behavior is outside the implemented guarantee.

Evaluation Scope

Compiler challenge success counts both valid-source acceptance and correct rejection of intentionally invalid source under the desired grammar. The nine-case mixed test is too small to establish broad compiler correctness. Unit tests cover precedence, associativity, unary operators, lexical failure and IR consistency. Further language features require new specifications, BERT supervision and formal checking contracts rather than another generic tree.

Evidence Boundaries and Rendering

Three Different Claims

Structure checked means a supplied or predicted DFA has a complete valid transition table, or a grammar/tree conforms to its explicit contract. Simulated means a trace follows that object for one input. Verified equivalence means the candidate DFA matches an independent reference for every word over the matching alphabet. These claims have different evidence requirements and remain distinct in result records.

An arbitrary English request normally lacks an independent formal reference. The application can show an unverified candidate preview and its trace. That view is useful for inspection and debugging, but the system does not relabel it as exact English correctness. Manual input is also clearly labelled and does not count as BERT prediction.

Sealed Export Objects

The DFA verification gate issues a sealed candidate/reference pair only after exact equivalence. The renderer accepts this evidence object rather than an arbitrary candidate dictionary. It rechecks the seal and equivalence, then reconstructs the requested trace. Tree exports likewise require production and yield checks. The contract prevents a rejected neural prediction from receiving an authoritative-looking verified video.

An incorrect candidate is never replaced by a known constructor output in the learned route. Tests deliberately supply a wrong BERT candidate for a known suffix benchmark and verify that the equivalence failure remains visible. Reference demonstrations are separately labelled, so using them is transparent rather than inflating learned accuracy.

Dynamic Visual Content

Graph layout derives nodes from actual states and edges from actual transitions. Start arrows, self-loops, reciprocal transitions and accepting rings reflect that structure. Trace frames derive the current state from simulation. Parse layout derives nodes and links from actual derivations. AST views derive their nodes from compiler lowering input. The visual system is reusable, while mathematical content is recomputed for each relevant request.

The final visualization uses stable reusable geometry and presentation components. Browser and HTTP checks verify current graph counts, changing trace acceptance, grammar tree growth and compiler behaviour against formal results. This protects representation fidelity without treating visual appearance as a proof of language interpretation.

Caching and Export Limits

Video cache keys depend on the validated payload and rendering options. A cache hit reuses a previously created file for the same evidence/input contract. Manim is optional and guarded by availability. Large graphs or trees may exceed readable rendering budgets even when formal checks succeed; the result retains the formal object and reports a visual limit rather than silently drawing an unrelated simplified structure.

Testing and Integration Verification

Formal and Neural-Decoder Tests

New predicate-based compiler tests enumerate binary words through length six and compare simulation against independent Python predicates for conjunction, union, negation, exact counts, count bounds, count modulo and total-length conditions. They cover empty input and overlapping literal patterns. Invalid modulus, invalid patterns, Boolean-as-integer arguments and state-budget violations are rejected.

A decoder test injects ideal logits for a composite plan with literals 00101 and 10. It verifies that learned-copy mechanics preserve leading zeros, respect full literal boundaries and reconstruct the intended typed tree. This tests the decoder independently of neural training quality. Dataset tests verify one partition per canonical language and no training-language overlap with the frozen comparison.

Application Regression Tests

Existing tests cover invalid structure, exact equivalence and shortest counterexamples, strict JSON, unverified manual previews, smoke-checkpoint rejection, no rule fallback, grammar reuse revision checks and compiler failures retaining actual productions. Head-checkpoint integration is checked through the same learned route used by the product. The full suite is rerun after final code changes, with results saved in the verification artifact.

Runtime Checks

The local server is restarted with the updated model dispatcher. HTTP checks exercise asynchronous jobs, independent-reference rejection, valid derivations, compiler tasks and available video export. Browser checks run the real React UI on desktop and mobile. They confirm suffix-01 produces three states, suffix-0101 produces five, a changed input changes acceptance, grammar inputs alter parse-tree size and compiler output follows source.

Presentation source fingerprints and browser checks protect graph and tree fidelity. Tests compare visible counts and traces with the formal result, verify that changed inputs update execution and check responsive overflow and console errors.

Recorded Verification Status

The final software suite passes 105 tests, and the frontend builds locally and in GitHub Actions. The local seven-case deployment check exercises genuine BERT, formal verification, compiler rejection and real video output. These integration checks are separate from held-out model accuracy.

The research comparison remains separate from software tests. Model semantic scores come from independently generated reference DFAs on the frozen cases. Software tests validate contracts and implementation behavior. Both are necessary, but neither should be substituted for the other in a project presentation.

Reproducibility and Local Operation

Starting the Existing Local Installation

```
.venv/bin/python scripts/run_local.py --port 8501
```

Open http://localhost:8501 after the server reports readiness. The launcher builds the frontend unless --skip-build is supplied. For an already built frontend, use .venv/bin/python scripts/run_local.py --port 8501 --skip-build. The Python environment must include the project dependencies, and Node is needed for rebuilding React. The project source configuration accepts Python 3.10 or later; the recorded local interpreter is Python 3.12.

Data and Model Reproduction

```
.venv/bin/python scripts/download_training_assets.py
.venv/bin/python -m theorybert.data.prepare --help
.venv/bin/python -m theorybert.data.expand --help
.venv/bin/python -m theorybert.data.plans
```

Review the preparation and expansion CLI options before rerunning corpus generation. The plan builder intentionally refuses to overwrite an existing frozen v2 comparison. For an independent later experiment, introduce a new version and freeze it before predictions. Reproducing the plan head requires the exact evaluated V3 encoder, not merely a fresh pretrained BERT download.

```
.venv/bin/python -m theorybert.models.semantic --epochs 60
.venv/bin/python scripts/compare_plan_model.py
.venv/bin/python -m pytest -q
```

The comparison script refuses to overwrite an existing selection. It is an audit boundary, not a command to repeatedly tune the same test. A clean reproduction should use isolated output paths and retain the protocol, dataset fingerprints and encoder artifact. Training is CPU-limited by code and can stop if resource thresholds are exceeded. Do not launch another training job alongside it on this machine.

Model Compatibility

The active pointer selects the BERT semantic-plan decoder. Its head and the grammar head require the exact fingerprinted V3 encoder. Explicit checkpoint selection must retain compatible configuration and source evidence. A changed encoder requires matching adaptation and evaluation rather than silent substitution.

Artifact Availability and Reporting

Source code, configuration, reports and dataset provenance are distributed through the GitHub project. Large FP32 inference assets are a separate GitHub Release bundle pinned by deploy/model-bundle.json. The Docker build verifies the archive and every member before installing it. Reproduction requires this exact encoder plus compatible task heads.

The two PDFs can be regenerated with .venv/bin/python docs/reports/build_reports.py after its report dependencies are available. They use embedded Times New Roman fonts and vector architecture diagrams. Their page renderings and text checks are retained as local QA artifacts.

Limitations and Failure Analysis

Language Understanding

The semantic-plan model remains imperfect on the fresh test under the frozen wording distribution. Typed validity does not eliminate operator confusion or numeric mistakes. Negation scope, multiple literals, inclusive/exclusive bounds, ambiguous descriptions and unseen terminology can cause incorrect plans. The model has no calibrated guarantee that a high-scoring output matches the English. A valid output can therefore be confidently wrong.

The task is binary and the plan vocabulary is finite. Requests for arbitrary alphabets, nonregular conditions, unrestricted long-count values or complex Boolean nesting are outside demonstrated coverage. The current implementation can still produce a valid in-scope interpretation for an out-of-scope request, so unsupported-intent detection is a research need. Type checks only validate the predicted object; they do not understand whether the request should have been rejected.

Evaluation and Generalization

The evaluation uses generated descriptions and one seed. It shares operator families with training and combines an architectural change with more supervision. Fresh test size is 120, with only 57 matched cases. The legacy regression test and retired challenge have been inspected repeatedly. These factors prevent broad claims about unrestricted user language, isolated architectural causality or statistically stable superiority across runs.

The full fresh score includes 63 cases beyond the direct structured interface bounds. It is useful for system coverage, but the matched subset is the appropriate direct-head comparison. Within-five success depends on a reference and cannot be promised for arbitrary free-form English. A general correctness percentage for the entire product would mix incompatible tasks and evidence types.

Grammar and Compiler Scope

Production exactness is sensitive to grammar spelling and order conventions. Alternative equivalent grammars may score differently, and general CFG equivalence is not checked. A successful derivation only proves membership under the predicted productions. Compiler support is limited to symbolic arithmetic and does not include a runtime, optimizer, machine backend, types or full language specification. Correct rejection is counted explicitly in negative compiler fixtures.

Performance and Operational Risks

The head is much smaller to train, but BERT remains the inference backbone and the new plan compiler adds work. The measured response-time cost is real. Resource guards use periodic snapshots and cannot prevent every brief pressure spike or unrelated background disk change. Historical checkpoints already consume substantial local storage, so another experiment should first budget its exact artifact requirements. No automatic deletion policy is introduced.

There has been no controlled educational user study, accessibility audit or internet-scale load test. The interface may help inspect formal behavior, but improved learning outcomes are unmeasured. These are future validation tasks rather than implied benefits of a visually polished system.

Integrated Contributions and Research Scope

A Single Research Argument

The inability of NLP systems to reliably support the translation and understanding of formal-language objects and their behaviour.

TheoryBERT responds with a BERT-centered translation layer, formal execution layer and interactive understanding layer. DFA, CFG and arithmetic compiler workspaces share that argument. The solution covers language interpretation, mathematical object construction, execution, evidence and presentation in one coherent application.

Complete Implemented Contribution

The neural layer provides contextual representations, learned plans, literal copying, direct structured automata and grammar-production decoding. The formal layer supplies construction, minimization, equivalence, simulation, bounded chart parsing, derivation checks, AST construction and symbolic IR. The application layer supplies asynchronous jobs, evidence ownership, caching, responsive studios, research information and reusable dynamic visualization.

The data and operational layer supplies source-pinned preprocessing, generated supervision, canonical-language partitions, challenge assets, manifests, training controls, checkpoint fingerprints, model packaging and a container deployment path. The companion dataset archive and public model release provide distinct distribution units for data and inference artifacts.

Project Uniqueness

The distinguishing feature is the integrated contract between learned language interpretation and checkable formal behaviour. The system does not stop at a fluent textual answer or a static graph. It exposes what BERT predicted, how the formal object behaves and which claim the evidence actually supports. This is an integration contribution grounded in established neural and formal methods.

Research Boundaries and Extension

Independent human-authored descriptions, several random seeds, calibrated abstention and educational user studies are needed to establish broader reliability and learning benefits. Richer alphabets, statement languages and other grammar classes require additional supervision and new formal contracts. Their absence does not change the central research problem, but limits the current implementation scope.

Paper Organization

The IEEE manuscript contains introduction, literature review, proposed solution architecture, key contributions, datasets, final evaluation, web and resource design, limitations and conclusion. This report expands every implemented subsystem and supplies provenance, source links, technology detail and reproducibility information.

Dataset Fingerprints and Exact Source Links

data/challenges/plan-comparison-v2.jsonl
254 rows; validation=134, test=120
SHA-256: 550b42a0566af360e46cea8c6f903c0d824aa805451c198f3549f18b352dc783

data/challenges/toc-compiler-v1.jsonl
165 rows; test=48, development=71, validation=46
SHA-256: ff33ccb8b859220015a605d404f75be78cfc7ca21075557805c0c375ad4bf73b

data/examples/challenges.jsonl
3 rows; challenge=3
SHA-256: 7bfad5cef0b9328c2c0a6927a572f9e17dfdddc2fde2b96d1fadd9650e5aaf0a

data/examples/sample.jsonl
261 rows; train=181, validation=39, test=41
SHA-256: 20220962f7d05b9b38fd24289cfaf07f45240b0bb1d5b0cac11b4fe5bcaa4e76

data/generated/english-dfa-expanded.jsonl
3,207 rows; train=2437, validation=391, test=379
SHA-256: 547e006cc5673ee20a46c6de512510eda7bdbf72d94507cd3a9a3c5b31e898d6

data/generated/english-dfa-robust.jsonl
6,738 rows; train=4924, validation=1173, test=641
SHA-256: bced1069877f875226ae34165c9d0c05f8b5a8fdd7cae7337bf28a4d0534b54a

data/generated/english-dfa-v3.jsonl
6,275 rows; train=4889, validation=745, test=641
SHA-256: a186d971fd113e723dc1ad2ca108d1b8d1c5220710398e0d29248399515e2172

data/generated/english-dfa.jsonl
1,785 rows; train=1363, validation=213, test=209
SHA-256: 7cb45cf3e2b4527aa67f35ee89c12decee6bde58b209d9fb3ab63a98e5bc7415

data/generated/english-plan-v1.jsonl
11,415 rows; train=9187, validation=1171, test=1057
SHA-256: a1f9ab1f907fb52015def4184a383d234ad059cd00d0188430ecb5978b9bf584

data/generated/grammar-robust.jsonl
456 rows; validation=85, train=371
SHA-256: 645f2fe6fc42674d88f843509fe17a3836969187716874f7b2febaae08c18d5e

artifacts/bert-grammar-pilot/dataset.jsonl
1,299 rows; train=999, validation=170, test=130
SHA-256: 2ea73a8b6fdc585d3199d6e9f11d690329367f36dc4dae96085058cb6f05c62a

artifacts/bert-grammar/dataset.jsonl
1,299 rows; train=999, validation=170, test=130
SHA-256: 2ea73a8b6fdc585d3199d6e9f11d690329367f36dc4dae96085058cb6f05c62a

artifacts/grammar-robust/dataset.jsonl
1,652 rows; validation=255, train=1267, test=130
SHA-256: 543c7ac4061d22194a69a493732da4818706e0dae224def6a192e0f08332d676

Exact External Files

kb13-descriptions.txt:
https://raw.githubusercontent.com/nicholaslocascio/deep-regex/096490db7f4b0394fbb46b914cb35a0aa1cba29c/datasets/KB13/src.txt

kb13-regex.txt:
https://raw.githubusercontent.com/nicholaslocascio/deep-regex/096490db7f4b0394fbb46b914cb35a0aa1cba29c/datasets/KB13/targ.txt

LICENSE.md:
https://raw.githubusercontent.com/nicholaslocascio/deep-regex/096490db7f4b0394fbb46b914cb35a0aa1cba29c/datasets/license.md

Bibliography

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, pp. 4171-4186, 2019.
https://aclanthology.org/N19-1423/

[2] N. Kushman and R. Barzilay, "Using Semantic Unification to Generate Regular Expressions from Natural Language," NAACL-HLT, pp. 826-836, 2013.
https://aclanthology.org/N13-1103/

[3] N. Locascio, K. Narasimhan, E. DeLeon, N. Kushman, and R. Barzilay, "Neural Generation of Regular Expressions from Natural Language with Minimal Domain Knowledge," EMNLP, pp. 1918-1923, 2016.
https://aclanthology.org/D16-1197/

[4] W. Tai, H. T. Kung, X. Dong, M. Comiter, and C.-F. Kuo, "exBERT: Extending Pre-trained Models with Domain-specific Vocabulary Under Constrained Training Resources," Findings of EMNLP, pp. 1433-1439, 2020.
https://aclanthology.org/2020.findings-emnlp.129/

[5] J. Herzig et al., "Unlocking Compositional Generalization in Pre-trained Models Using Intermediate Representations," arXiv:2104.07478, 2021.
https://arxiv.org/abs/2104.07478

[6] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," EMNLP, pp. 9895-9901, 2021.
https://aclanthology.org/2021.emnlp-main.779/

[7] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList," ACL, pp. 4902-4912, 2020.
https://aclanthology.org/2020.acl-main.442/

[8] I. Oren, J. Herzig, N. Gupta, M. Gardner, and J. Berant, "Improving Compositional Generalization in Semantic Parsing," Findings of EMNLP, 2020.
https://aclanthology.org/2020.findings-emnlp.225/

[9] N. Locascio, DeepRegex code and KB13 dataset mirror, revision 096490db7f4b0394fbb46b914cb35a0aa1cba29c, MIT dataset license.
https://github.com/nicholaslocascio/deep-regex/tree/096490db7f4b0394fbb46b914cb35a0aa1cba29c/datasets/KB13

[10] Google BERT community / Hugging Face, bert-base-uncased model card and weights, revision 86b5e0934494bd15c9632b12f734a8a67f723594.
https://huggingface.co/google-bert/bert-base-uncased

[11] NLTK Project, nltk.parse.earleychart: Incremental chart parsing documentation.
https://www.nltk.org/api/nltk.parse.earleychart.html

[12] Cornell CS 4120, "Automating Lexical Analysis," course notes, Spring 2023.
https://www.cs.cornell.edu/courses/cs4120/2023sp/notes/leximpl/

[13] TheoryBERT project repository, local implementation and recorded experiments. The local working tree contains changes that may not yet be published.
https://github.com/samyaksrijan/theoryBERT

Public sources checked on 29 October 2026. Original research is cited for context and method inspiration. All project experiment figures come from the local manifests and completed run artifacts. Inference weights are distributed through the versioned model release; the source repository contains the pinned bundle manifest.

Additional Local Evidence

artifacts/plan-promotion-protocol.json; artifacts/plan-comparison/selection.json; artifacts/plan-comparison/summary.json; artifacts/bert-plan/best/summary.json; artifacts/plan-verification.json; artifacts/challenge-comparison.json; artifacts/report-data-inventory.json; artifacts/plan-visual-check.json.

Implementation Map and Glossary

Source location	Responsibility
src/theorybert/models/theorybert.py	Pretrained encoder, adapted features and direct DFA heads.
src/theorybert/models/semantic.py	Plan head, literal pointers, decoding and guarded training.
src/theorybert/models/predictor.py	Checkpoint dispatch and compatible encoder injection.
src/theorybert/models/grammar.py	Production-token decoder, grammar corpus and training.
src/theorybert/formal/plan.py	Typed plan validation and exact DFA construction.
src/theorybert/formal/dfa.py	Validation, trace, minimization and equivalence.
src/theorybert/formal/cfg.py	Bounded parsing and derivation checks.
src/theorybert/formal/compiler.py	Arithmetic lexing, AST and checked symbolic IR.
src/theorybert/formal/analysis.py	Machine/grammar facts derived from current objects.
src/theorybert/web.py	API jobs, active artifacts, cache and provenance.
frontend/src/App.jsx	Workspaces, inputs, evidence, playback and settings.
frontend/src/components/Diagram.jsx	Reusable graph/tree visual components.
src/theorybert/rendering/	Geometry, sealed rendering payloads and Manim scenes.
src/theorybert/data/	Generation, import, augmentation, challenges and plans.
scripts/compare_plan_model.py	Frozen validation selection and later test evaluation.

Recorded Local Technology Versions

Technology	Version	Technology	Version
Python	3.12.4	torch	2.14.1
transformers	4.57.6	starlette	1.9.0
uvicorn	0.54.0	nltk	3.10.4
numpy	2.5.4	manim	0.22.0
streamlit	1.66.1	react	19.3.1
react-router-dom	7.18.4	vite	8.3.5
motion	14.6.0	lucide-react	1.61.0

Glossary

DFA: deterministic finite automaton. CFG: context-free grammar. AST: abstract syntax tree. IR: intermediate representation. EOS: end-of-sequence marker. Canonical language identity: minimal reachable DFA renamed in fixed BFS order. First-candidate equivalence: exact semantic correctness before reference-assisted search. Within-five: a correct candidate found with an independent reference. Plan: model-produced typed regular-language expression. Frozen encoder: weights reused without gradient updates. Preview: inspectable candidate without independent equivalence evidence. Sealed evidence: a verified formal object authorized for the corresponding rendering contract.

The companion manuscript presents the research argument, while this report supplies the detailed implementation and operational record. The dataset archive supplies local supervision and provenance. Reproducing the learned outputs also requires the fingerprinted evaluated encoder and compatible trained heads.

Expanded Review of the Supplied Literature

Attention, Pretraining and Adaptation

Vaswani et al. [14] provide the attention-based Transformer foundation. RoBERTa [15] emphasizes careful control of pretraining data and optimization. Together they support treating architecture, supervision and training condition as separate experimental variables. TheoryBERT uses the original base uncased BERT lineage, not RoBERTa, and its direct-versus-plan comparison changes both representation and supervised data.

BioBERT [16] demonstrates domain-specific continued pretraining in biomedicine. The more recent ModernBERT legal adaptation study [24] supplies another domain-adaptation example. These works motivate domain sensitivity but do not imply that biomedical or legal data would improve automata reasoning. Neither corpus nor checkpoint family is used in the local experiments. A formal-language continued-pretraining study would need its own corpus, cost budget and held-out evaluation.

Vocabulary, Distillation and Compute

The publisher-indexed Efficient Token Augmented Fine-Tuning record [19] identifies token augmentation as an alternative to expensive continued pretraining. Its full experimental text was not accessible in this research pass, so the report does not infer its initialization recipe or reproduce its performance claims. The supplied venue acronym is corrected to CSDE. This distinction matters because a valid citation is not evidence that a method was implemented here.

Ryu and Lee [20] study adversarial adaptation with distillation for domain shift. Linder's KTH thesis [21] studies vocabulary expansion through self-distillation across tokenizations. These are plausible future controls for preserving old behavior while learning domain terms. The completed plan run uses supervised token and literal-pointer losses, not teacher-distribution distillation. Its efficiency result comes from a small head over frozen features.

Formal Languages and Semantic Parsing

Deletang et al. [17] study neural generalization across formal-language task classes. Strobl et al. [18] organize transformer-expressivity results under their assumptions. Their relevance is a warning: success on finite generated examples does not establish unrestricted recognition or compositional generalization. TheoryBERT delegates execution to classical formal algorithms after a bounded learned interpretation.

Locascio et al. [3] remain directly relevant to natural-language regex generation and the public-data lineage. Xu et al. [25] provide a separate semantic-parsing example over Wikidata. The latter uses another task, data source and model family. TheoryBERT does not use WikiWebQuestions, Wikidata, LLaMA or GPT as a hidden prediction path.

Peripheral Context and Scope

The visual-language study by Alper et al. [22] concerns implicit visual knowledge in language representations. It does not validate drawing DFA graphs. CyMapNER [23] concerns domain-specific named entity recognition, not automata construction. Both are verified references and useful broader context, but neither supplies implementation evidence for this prototype. None of the extra literature datasets is included in the local training totals.

Supplemental Bibliography

These numbers continue the comprehensive report bibliography. The eight-page manuscript independently numbers its selected sources by first citation order.

[14] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems, vol. 30, 2017.
https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[15] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019.
https://arxiv.org/abs/1907.11692

[16] J. Lee et al., "BioBERT: a pre-trained biomedical language representation model for biomedical text mining," Bioinformatics, vol. 36, no. 4, pp. 1234-1240, 2020.
https://academic.oup.com/bioinformatics/article/36/4/1234/5566506

[17] G. Deletang et al., "Neural Networks and the Chomsky Hierarchy," ICLR, 2023; arXiv:2207.02098.
https://arxiv.org/abs/2207.02098

[18] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin, "What Formal Languages Can Transformers Express? A Survey," TACL, vol. 12, pp. 543-561, 2024.
https://aclanthology.org/2024.tacl-1.30/

[19] L. J. Weber, K. J. Ramalingam, M. Beyer, A. Kirchheim, and A. Zimmermann, "Efficient Token Augmented Fine-Tuning," IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE), 2023, doi:10.1109/CSDE59766.2023.10487678.
https://doi.org/10.1109/CSDE59766.2023.10487678

[20] M. Ryu and K. Lee, "Knowledge Distillation for BERT Unsupervised Domain Adaptation," arXiv:2010.11478, 2020.
https://arxiv.org/abs/2010.11478

[21] M. R. Linder, "Vocabulary Expansion of Large Language Models via Self-Distillation," M.S. thesis, KTH Royal Institute of Technology, Jan. 2026; arXiv:2508.15807v2.
https://arxiv.org/pdf/2508.15807

[22] M. Alper, M. Fiman, and H. Averbuch-Elor, "Is BERT Blind? Exploring the Effect of Vision-and-Language Pretraining on Visual Language Understanding," CVPR, 2023.
https://openaccess.thecvf.com/content/CVPR2023/papers/Alper_Is_BERT_Blind_Exploring_the_Effect_of_Vision-and-Language_Pretraining_on_CVPR_2023_paper.pdf

[23] J. Zhang, F. Shi, C. Xu, Y. Li, X. Yin, and M. Ge, "CyMapNER: a named entity recognition model for cyberspace surveying and mapping domain," Cybersecurity, vol. 9, art. 146, 2026.
https://link.springer.com/article/10.1186/s42400-026-00578-3

[24] D. Stammbach and P. Henderson, "Legal Domain Adaptation of Modern BERT Models," arXiv:2606.28538, 2026.
https://arxiv.org/abs/2606.28538

[25] S. Xu et al., "Fine-tuned LLMs Know More, Hallucinate Less with Few-Shot Sequence-to-Sequence Semantic Parsing over Wikidata," EMNLP, pp. 5778-5791, 2023.
https://aclanthology.org/2023.emnlp-main.353/

TAFT metadata and abstract were checked in the publisher-indexed proceedings; the full IEEE paper was inaccessible. The audit on the following pages distinguishes verified metadata, corrected details and project relevance. Literature-only datasets and models are not project training inputs.

Reference Audit: Supplied Items 1-8

Supplied item numbers refer to the bibliography provided by the user. Report reference numbers refer to the comprehensive report. Verification checks identity and publication metadata; it does not imply a reproduced experiment.

Supplied [1] / report [14]: Verified
A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems, vol. 30, 2017.
The official NeurIPS proceedings confirm the 2017 Transformer paper. Used as the attention foundation; its translation data are not local project datasets. Primary record

Supplied [2] / report [1]: Verified
J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, pp. 4171-4186, 2019.
NAACL-HLT 2019 record confirms authors and title. This is the actual encoder lineage used through the pinned base uncased artifact. Primary record

Supplied [3] / report [15]: Verified, contextual
Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019.
The 2019 arXiv record is correct. RoBERTa is discussed as pretraining-control context; no RoBERTa checkpoint or corpus is used here. Primary record

Supplied [4] / report [4]: Verified
W. Tai, H. T. Kung, X. Dong, M. Comiter, and C.-F. Kuo, "exBERT: Extending Pre-trained Models with Domain-specific Vocabulary Under Constrained Training Resources," Findings of EMNLP, pp. 1433-1439, 2020.
Findings of EMNLP 2020 confirms the citation. The project extension is exBERT-inspired and is not an exact architecture or biomedical-training reproduction. Primary record

Supplied [5] / report [16]: Verified, full title restored
J. Lee et al., "BioBERT: a pre-trained biomedical language representation model for biomedical text mining," Bioinformatics, vol. 36, no. 4, pp. 1234-1240, 2020.
The full title ends with 'for biomedical text mining'. The issue year is 2020 although advance online publication was September 2019. No BioBERT weights or biomedical corpora are used. Primary record

Supplied [6] / report [20]: Verified as the 2020 preprint
M. Ryu and K. Lee, "Knowledge Distillation for BERT Unsupervised Domain Adaptation," arXiv:2010.11478, 2020.
The arXiv version lists Minho Ryu and Kichun Lee. A later 2022 journal version also lists Geonseok Lee; these are different bibliographic versions. Distillation is not implemented in this project. Primary record

Supplied [7] / report [19]: Venue acronym corrected; abstract-level verification
L. J. Weber, K. J. Ramalingam, M. Beyer, A. Kirchheim, and A. Zimmermann, "Efficient Token Augmented Fine-Tuning," IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE), 2023, doi:10.1109/CSDE59766.2023.10487678.
The publisher index identifies IEEE CSDE 2023, not APSCDE. Title and five authors match. DOI is 10.1109/CSDE59766.2023.10487678. Full paper access was unavailable, so detailed experimental claims are not used. Primary record

Supplied [8] / report [21]: Thesis verified; version clarified
M. R. Linder, "Vocabulary Expansion of Large Language Models via Self-Distillation," M.S. thesis, KTH Royal Institute of Technology, Jan. 2026; arXiv:2508.15807v2.
The PDF cover identifies KTH, the exact supplied thesis title and 14 January 2026. The arXiv landing title adds Kullback-Leibler-Based; v1 is August 2025 and v2 is January 2026. No self-distillation run is claimed here. Primary record

Checked 29 October 2026. The authors of cited works are not the authors of this project. The project author/institution block remains omitted because names were not supplied.

Reference Audit: Supplied Items 9-15

Supplied item numbers refer to the bibliography provided by the user. Report reference numbers refer to the comprehensive report. Verification checks identity and publication metadata; it does not imply a reproduced experiment.

Supplied [9] / report [22]: Verified, peripheral
M. Alper, M. Fiman, and H. Averbuch-Elor, "Is BERT Blind? Exploring the Effect of Vision-and-Language Pretraining on Visual Language Understanding," CVPR, 2023.
The CVPR author paper confirms the title and authors. It studies visual-language knowledge, not the correctness of generated DFA diagrams. It is contextual only; this project has no vision-language pretraining. Primary record

Supplied [10] / report [23]: Verified, title expanded
J. Zhang, F. Shi, C. Xu, Y. Li, X. Yin, and M. Ge, "CyMapNER: a named entity recognition model for cyberspace surveying and mapping domain," Cybersecurity, vol. 9, art. 146, 2026.
The publisher gives 'a named entity recognition model', Cybersecurity volume 9, article 146, published 2 April 2026. It concerns cyberspace NER, not automata inference. Its datasets are not project inputs. Primary record

Supplied [11] / report [24]: Verified 2026 preprint
D. Stammbach and P. Henderson, "Legal Domain Adaptation of Modern BERT Models," arXiv:2606.28538, 2026.
arXiv:2606.28538 confirms Stammbach and Henderson. It adapts ModernBERT in law, whereas this project uses original base uncased BERT and supervised formal outputs. Legal corpora and checkpoints are not used. Primary record

Supplied [12] / report [3]: Verified; proceedings version used
N. Locascio, K. Narasimhan, E. DeLeon, N. Kushman, and R. Barzilay, "Neural Generation of Regular Expressions from Natural Language with Minimal Domain Knowledge," EMNLP, pp. 1918-1923, 2016.
EMNLP 2016, pages 1918-1923, DOI 10.18653/v1/D16-1197. This is the directly relevant natural-language regex paper and repository lineage for the filtered KB13 mirror. Primary record

Supplied [13] / report [17]: Verified
G. Deletang et al., "Neural Networks and the Chomsky Hierarchy," ICLR, 2023; arXiv:2207.02098.
The author arXiv PDF states ICLR 2023; its initial preprint is 2022. Its generalization study motivates careful formal-task boundaries, not a claim that this prototype reproduces the study. Primary record

Supplied [14] / report [18]: Verified
L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin, "What Formal Languages Can Transformers Express? A Survey," TACL, vol. 12, pp. 543-561, 2024.
TACL volume 12, pages 543-561, 2024, DOI 10.1162/tacl_a_00663. The survey highlights architecture-dependent assumptions; it does not prove that this BERT system solves all formal languages. Primary record

Supplied [15] / report [25]: Verified, contextual
S. Xu et al., "Fine-tuned LLMs Know More, Hallucinate Less with Few-Shot Sequence-to-Sequence Semantic Parsing over Wikidata," EMNLP, pp. 5778-5791, 2023.
ACL confirms EMNLP 2023, pages 5778-5791, DOI 10.18653/v1/2023.emnlp-main.353. Its Wikidata semantic parser is relevant background; no WikiWebQuestions/Wikidata data or LLaMA/GPT route is used here. Primary record

Checked 29 October 2026. The authors of cited works are not the authors of this project. The project author/institution block remains omitted because names were not supplied.